

Reviewer Pack — Minimal Local Induction Dimension and Communication Complexi...

Entry f9dada324955 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 16:00:07 UTC

Minimal Local Induction Dimension and Communication Complexity Rank Variance Bound

Entry ID: f9dada324955 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 16:00:07 UTC

1. Conjecture

Field A (mathematical branch): Descriptive Set Theory (Local Induction Dimension) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given n -communication protocol P , the local induction dimension (LID) of its input space is upper-bounded by a constant multiple of the variance in the communication complexity rank among all possible inputs with size n , i.e., $LID(P) = O(\kappa * \text{CommRankVar}(P))$, where κ is a constant.

Rationale (proposer's reasoning):

The Local Induction Dimension provides a measure of the complexity of a set in terms of its ability to be approximated by simpler sets. If communication complexity rank varies significantly with inputs, it suggests that there may be hidden patterns or structures within the protocol's input space that contribute to its complexity. This conjecture aims to bridge descriptive set theory and communication complexity by relating these two concepts.

Taxonomy category: LocalInductionDimension (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b2e8b9390b56519d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if the Pearson correlation coefficient between $LID(P)$ and $\text{CommRankVar}(P)$ is ≥ 0.8 for all $n \leq 40$, and no seed produces an LID value $> \kappa * \text{mean}(\text{CommRankVar}(P)) + 3\sigma$, where κ is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Constants
MAX_N = 40
NUM_INSTANCES_PER_SEED = 30
CONJECTURE_SUPPORT_THRESHOLD = 0.8
CONSTANT_MULTIPLE = 1.5

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def calculate_comm_rank(protocol):
        return sum(p) / len(p)

    def calculate_comm_rank_var(protocol):
        comm_ranks = [calculate_comm_rank(p) for p in protocol]
        mean = sum(comm_ranks) / len(comm_ranks)
        var = sum((x - mean) ** 2 for x in comm_ranks) / len(comm_ranks)
        return var

    def run_protocol(n):
        # Generate a random n-communication protocol
        protocol = [[random.randint(1, 10) for _ in range(n)] for _ in range(NUM_INSTANCES_PER_SEED)]
        lid = sum(p[0] for p in protocol) / len(protocol)
        comm_rank_var = calculate_comm_rank_var(protocol)
        return lid, comm_rank_var

    results = []
    n_values = [5, 10, 15, 20, 30, 40]

    for n in n_values:
        lid, comm_rank_var = run_protocol(n)
        results.append({
            "n": n,
            "lid": lid,
```

```

        "comm_rank_var": comm_rank_var
    })

mean_lid = sum(result["lid"] for result in results) / len(results)
std_lid = math.sqrt(sum((result["lid"] - mean_lid) ** 2 for result in results) / len(results))

correlation_coefficient = sum((result["lid"] - mean_lid) * (result["comm_rank_var"] - mean_comm_rank_var)
                               for result in results) / (len(results) * std_lid * mean_comm_rank_var)

if correlation_coefficient < CONJECTURE_SUPPORT_THRESHOLD:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": False,
        "counterexample": "Correlation coefficient is below threshold"
    }

if any(result["lid"] > CONSTANT_MULTIPLE * mean_comm_rank_var + 3 * std_lid for result in results):
    return {
        "metric_name": "lid_bound",
        "metric_value": max(result["lid"] for result in results),
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": False,
        "counterexample": "LID exceeds bound"
    }

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= CONJECTURE_SUPPORT_THRESHOLD:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if not r["conjecture_holds"])

```

```

print(f"RESULT: FALSIFIED counterexample='Correlation coefficient below threshold or LID e
else:
print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b6d7101a.py", line 95, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b6d7101a.py", line 47, in run_trial
    lid, comm_rank_var = run_protocol(n)
                        ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b6d7101a.py", line 40, in run_protocol
    comm_rank_var = calculate_comm_rank_var(protocol)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b6d7101a.py", line 31, in calculate_com
    comm_ranks = [calculate_comm_rank(p) for p in protocol]
                  ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b6d7101a.py", line 28, in calculate_com
    return sum(p) / len(p)
           ^
NameError: name 'p' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14241
2	preregistration	ollama_remote	glm4:latest	0	0	10054
3	novelty	ollama_remote	glm4:latest	0	0	9081
4	novelty	ollama_remote	glm4:latest	0	0	8586

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35823
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13740
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23467
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12950
9	judge	ollama_remote	glm4:latest	0	0	24487

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 152430 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f9dada324955.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f9dada324955.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f9dada324955.tar.gz` (if generated)